

芯片安全软件方案

1. 概述

本方案面向 NGU800 的全生命周期安全能力建设，覆盖安全启动、固件验签与解密、安全服务接口、生命周期与安全 Debug、设备远程认证、制造灌装、RMA 以及板级/OOB 安全边界。

1.1 总体目标

- 建立以 eHSM 为唯一 Root of Trust 的安全启动与安全服务体系。
- 保证 FMC(SEC1)、GSP(SEC2) 和关键 Runtime 固件在执行前完成验签、解密、反回滚和 release policy 检查。
- 将 Host/BMC 等外部管理实体限定为不可信请求方或链路承载方；将 OOB MCU 纳入板级安全边界，作为受控带外刷写、电源复位和管理代理，但不进入 eHSM Root of Trust。
- 采用单 FMC 固定分区，将 FMC 损坏恢复能力转移到 OOB MCU + QSPI 板级恢复通道；允许 Host/BMC/运维工具通过 OOB 通路重发 FMC 包并触发重刷写。
- 建立可导出的 measurement table 和 attestation report，使验证方能够判断设备身份、固件版本、生命周期、Debug 状态、安全启动状态和 OOB 恢复状态。
- 建立制造灌装、USER 冻结、密钥轮换、RMA 和审计流程，支撑量产部署和售后闭环。

1.2 核心设计结论

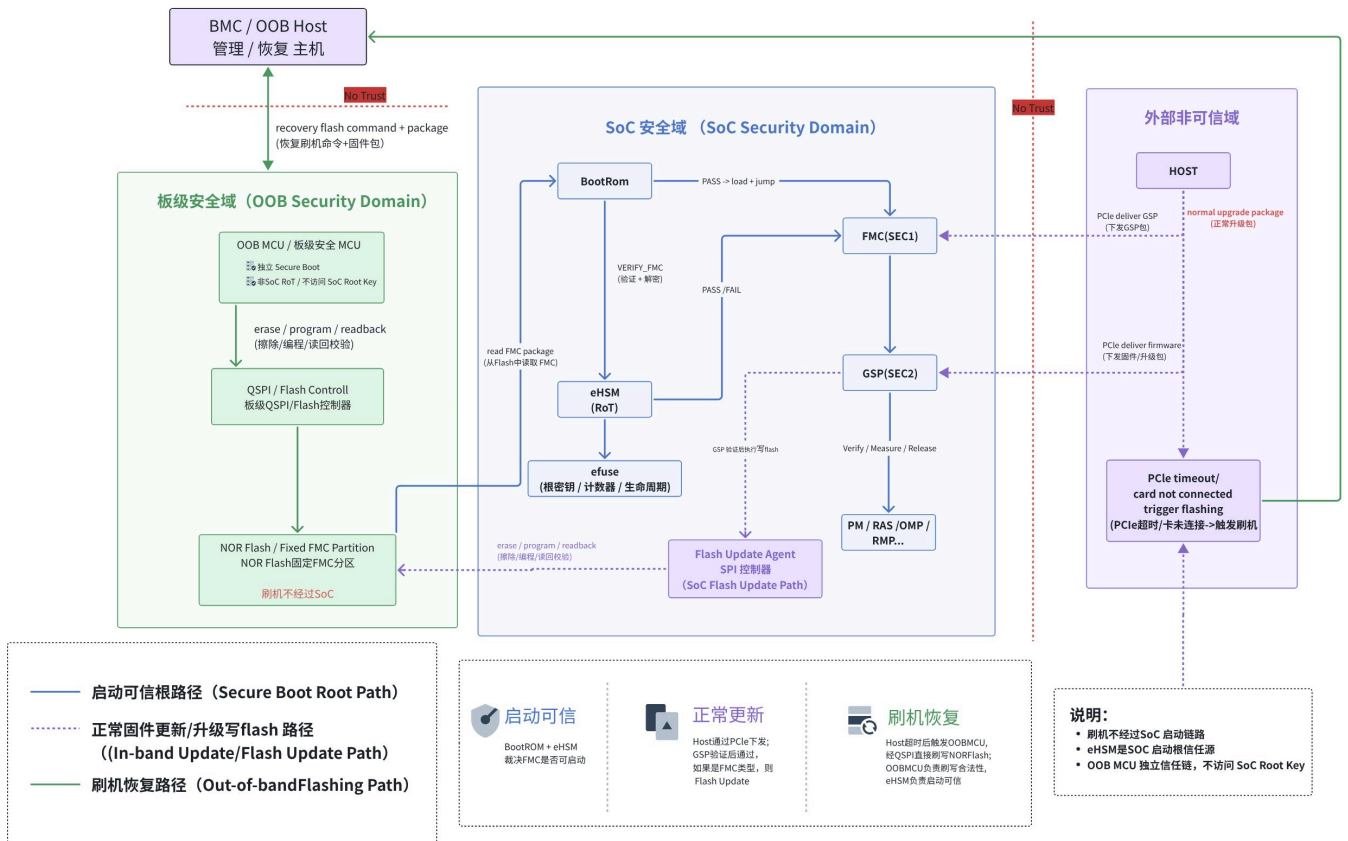
项目	结论
Root of Trust	eHSM 是 SoC 唯一 Root of Trust；Root Secret、key usage、counter、lifecycle、debug auth 和 at 均由 eHSM 控制。
BootROM 角色	BootROM 是最早启动编排者，负责最小初始化、读取固定 FMC 分区、调用 eHSM verify/decrypt、最 measurement 和跳转；不实现复杂密码库，不持有私钥。
First Mutable Stage	FMC 是 SEC1，对应 fmc-bm.bin 来自本地 NOR Flash 固定分区，不通过 Host PCIe 正常更新链路下发
Runtime Security Control	GSP 是 SEC2，对应 gsp-bm.bin，负责 Host/OOB 请求收敛、后续固件验证、measurement、attest RMA 和设备安全灌装流程编排。
正常固件更新	SoC 正常运行后，Host 通过 PCIe 下发 GSP 或 Runtime protected package；FMC/GSP 调 eHSM 完 必要解密、measurement 和 release。该流程可直接刷写 NOR Flash 固定 FMC 分区。
刷机恢复	Host 发现 PCIe 枚举失败、卡连接超时或心跳超时后，触发 BMC/OOB Host -> OOB MCU 刷机流程；信任链验证恢复刷机命令和包，并通过 QSPI 直接擦写 NOR Flash 固定 FMC 分区。
Host 边界	Host 是外部不可信请求方。正常更新时只负责传输 GSP/Runtime 包，不拥有 release 权；刷机恢复时 MCU，不直接访问 SoC Root、eHSM、eFuse、Secure SRAM 或 release 寄存器。
OOB MCU 边界	OOB MCU/板级安全 MCU 纳入板级安全边界，具备独立 Secure Boot、独立信任链和刷机授权校验能 过 SoC。OOB MCU 不作为 SoC RoT，不访问 SoC Root Key，不替代 eHSM 启动裁决。
FMC 防变砖模型	FMC 损坏、刷写中断或单分区不可启动时，通过 Host/BMC 触发 OOB MCU 重刷固定 FMC 分区；Boc 启动时验证并放行。
固件保护	FMC、GSP 正式路径强制签名 + 加密；PM/RAS/Codec 等关键 Runtime 固件默认签名 + 加密，产品白 固件可采用 signature-only。
镜像格式	采用 eHSM native secure image package；NGU 项目级 metadata 放入被 eHSM 认证/解密保护的 N manifest。
算法策略	支持国密和国际算法栈；算法 authority 来自 eHSM control field，manifest/report 只记录 expected
Attestation	采用静态设备证明证书 + measurement report。报告必须覆盖 nonce、measurement、lifecycle、c boot、rollback、image protection policy 和必要的 OOB/恢复状态。
Debug/RMA	USER/PROD 默认关闭 Debug；Debug/RMA 必须 challenge-response、scope、expire 和 audit。

2. 安全总体架构

2.1 逻辑架构

总体安全架构图

总体安全架构图



- 安全启动根路径位于 SoC 域内：NOR Flash 固定 FMC 包 -> BootROM -> eHSM -> FMC。
- 正常固件更新路径位于 SoC 正常运行态：Host 经 PCIe 下发 GSP/Runtime 包，FMC/GSP 调 eHSM 验证。
- 刷机恢复路径位于板级 OOB 域：Host/BMC 触发 OOB MCU，OOB MCU 通过 QSPI 直接写 NOR Flash。
- OOB MCU 具备独立信任链，只负责刷写合法性和写后校验，不访问 SoC Root Key。
- eHSM 是 SoC RoT，负责下一次启动时判断 FMC 是否可执行。

2.2 角色职责

角色	软件职责	安全边界
BootROM	最小平台初始化、读取 secure boot/lifecycle 配置、从固定 NOR Flash 地址读取 FMC package、调用 eHSM 验证和解密、做最小 manifest 检查、记录 measurement、跳转 FMC。	不持有私钥，不直接实现信任 Host 输入，不写 eFuse 和 fallback。
FMC(SEC1)	建立基础安全运行环境、初始化 Host 通道、接收 GSP 包、调用 eHSM 验证 GSP、向 GSP 转交运行期安全控制面。	只在 eHSM PASS 和 manifest PASS 后放行 GSP。
GSP(SEC2)	后续固件验证、release 控制、measurement 维护、SPDM Responder、Debug/RMA、Provisioning 编排。	是运行期安全控制面，负责安全收敛。
eHSM	Root Secret 使用、签名验证、解密、KDF/key unwrap、counter、lifecycle、debug auth、attestation signing。	私钥和根材料不出 eHSM，只通过接口连调用。
Host/Driver	传输 GSP/Runtime 固件包，发起 SPDM、状态查询、升级、Debug/RMA 请求；FMC 恢复时通过 BMC/OOB 通道重发 FMC recovery package。	不可信，不参与信任链，不直接写 eHSM/eFuse/Secure SRAM。
BMC/OOB Host	发起带外管理、运维、升级、FMC 恢复和电源控制请求。	不可信外部请求方，不写 eFuse；FMC 恢复包必须经过安全边界权检查。
OOB MCU/板级安全 MCU	纳入板级安全边界；承载带外代理、电源/复位、状态采集、JTAG MUX 控制和 QSPI 刷写 NOR Flash 固定 FMC 分区。	必须自身安全启动和受信任，通过 eHSM Root of Trust，直接判定 FMC 可执行。

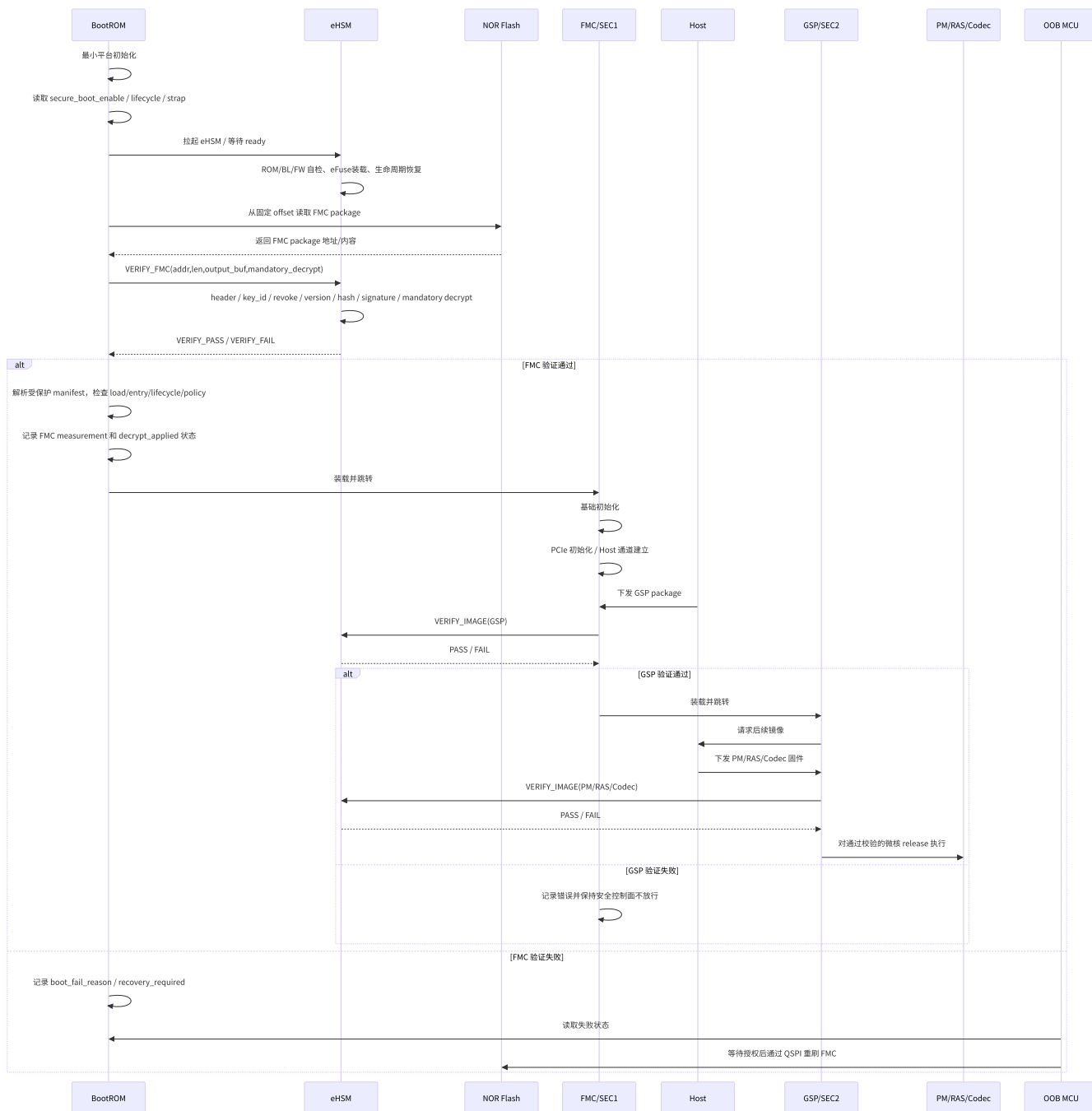
2.3 信任边界

- eHSM、eFuse、Secure SRAM、SEC 执行区、证书/策略区属于 SoC 安全域。
- OOB MCU 属于板级安全边界内的受控执行体，安全级别高于 BMC/Host 链路，但不等于 SoC Root of Trust。
- Host PCIe inbound、BMC/OOB Host、SMBus/I2C/I3C、JTAG、UART、管理 DMA 都属于不可信输入面。
- Host 和 OOB 只能访问普通 staging buffer、普通状态寄存器、OOB recovery 状态和白名单接口。
- release、debug enable、lifecycle、flash secure write、rollback counter、key revoke 等高危控制面必须由 SEC/eHSM 控制。
- 固定 FMC 分区可由 OOB MCU 通过 QSPI 在授权维护流程下重写，但重写后的 FMC 必须经过 BootROM + eHSM 重新验证后才可执行。

3. 安全启动

3.1 安全启动详细时序

安全启动详细时序图



步骤说明：

1. BootROM early 阶段：

- 上电或复位后，BootROM 先做最小平台初始化，只读取启动所需的 `secure_boot_enable`、lifecycle、strap 和 control field 等配置。该阶段不信任 Host 输入，也不进入复杂更新逻辑。

2. 拉起 eHSM 并等待 eHSM ready：

- BootROM 必须等待 eHSM 完成 ROM/BL/FW 自检、eFuse 装载和生命周期恢复。eHSM 未 ready 或 mailbox 超时都属于受控失败，BootROM 不允许绕过 eHSM 继续执行 FMC。

3. 读取固定 FMC 包：

- a. BootROM 只从 NOR Flash 的固定 FMC 分区读取 `FMC(SEC1)` secure package。

4. FMC verify/decrypt 裁决：

- a. BootROM 调用 eHSM `VERIFY_FMC(package_addr, output_buf)`，由 eHSM 完成 header、key、revoke、rollback、signature 和强制解密检查。正式路径中 FMC 必须签名 + 加密，验证或解密失败时 BootROM 停止跳转，可选记录诊断状态并等待复位或 OOB 刷机恢复。

5. FMC manifest 与跳转：

- a. eHSM 返回 PASS 后，BootROM 对 NGU protected manifest 做最小策略检查，包括 load 地址、entry 地址、lifecycle 约束和 measurement 记录。检查通过后，BootROM 将控制权交给 FMC；至此启动链进入 `BootROM -> FMC(SEC1, fixed NOR Flash partition)`。

6. FMC 建立 Host 通道并验证 GSP：

- a. FMC 完成基础初始化并建立 Host PCIe 通道。Host 只负责把 `GSP(SEC2)` protected package 传输到 staging buffer，不拥有 release 权；正式路径中 GSP 也必须签名 + 加密，FMC 调 eHSM `VERIFY_IMAGE(GSP)` 完成验签、解密、rollback、revoke 和 manifest policy 检查，PASS 后才 release GSP。

7. GSP 验证 Runtime 固件：

- a. GSP 接管运行期安全控制面后，继续接收 Host 下发的 PM/RAS/Codec/Other Runtime FW package，并调用 eHSM 完成 `VERIFY_IMAGE`、measurement 和 release policy 检查。Runtime 固件默认签名 + 加密，只有产品白名单允许的非敏感固件才可采用 signature-only。

8. 失败处理原则：

- a. 任一关键镜像验证失败都不允许降级为继续执行。GSP 验证失败时 FMC 向 Host 返回错误并等待重新下发；Runtime 验证失败时 GSP 拒绝 release 对应微核；FMC 验证失败时 BootROM 留在 halt / wait reset 状态。

9. OOB 刷机恢复关系：

- a. FMC 验证失败后的恢复不由 BootROM 自动 fallback 完成，而是等待 Host/BMC 触发 OOB MCU。
- b. OOB MCU 通过自身独立信任链检查 recovery flash command + package，经 QSPI 重刷 NOR Flash 固定 FMC 分区；重刷后的 FMC 仍需在下次启动中由 BootROM + eHSM 完成最终 verify/decrypt 裁决。

整体启动链收敛为：

代码块

```
1 BootROM -> FMC(SEC1, fixed NOR Flash partition) -> GSP(SEC2, Host 下发) -> PM/RAS/Codec/Other Runtime FW
```

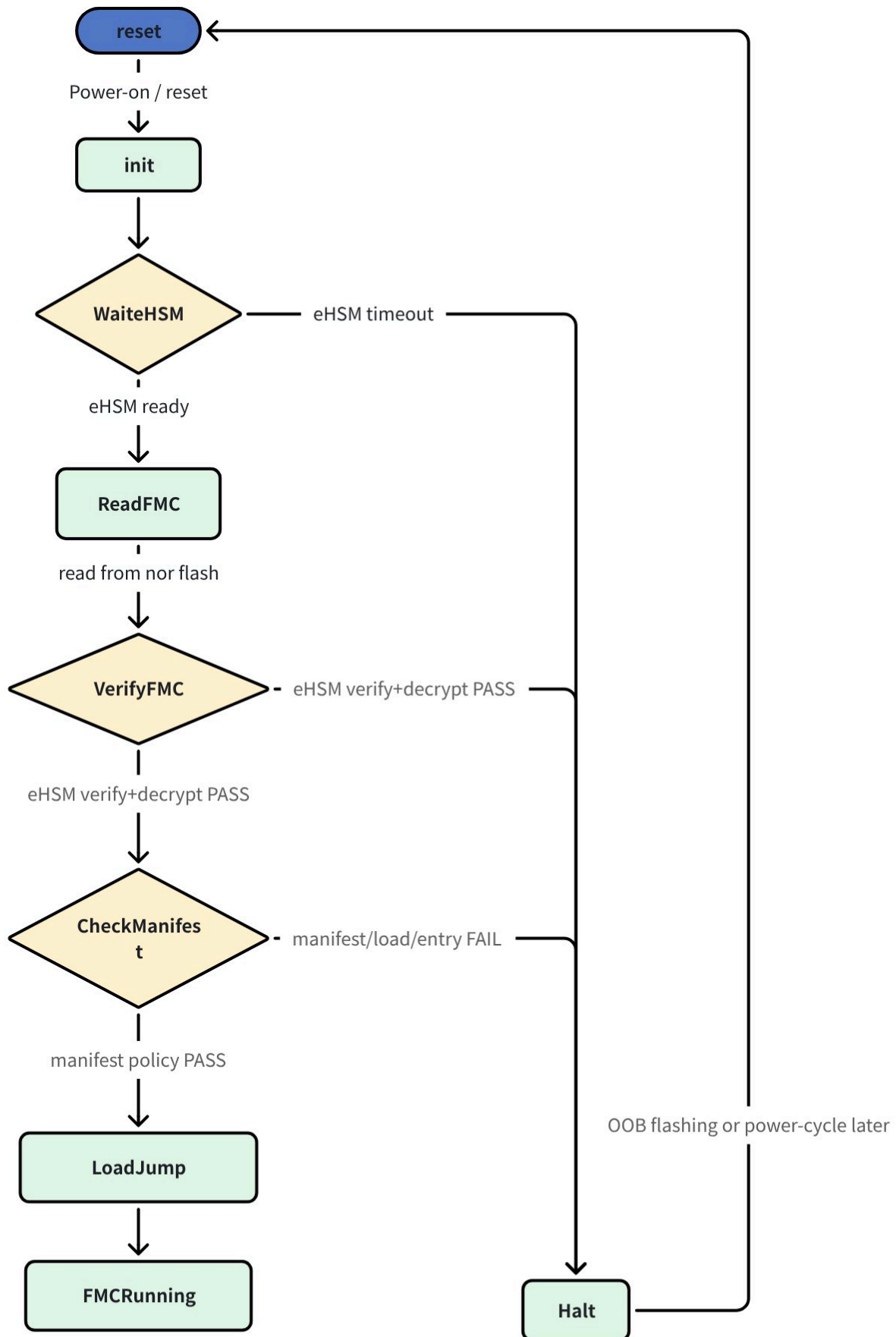
3.2 镜像策略

镜像	来源	验证发起	验证执行	强制保护
FMC(SEC1)	本地 NOR Flash 固定分区	BootROM	eHSM	强制签名 + 加密 + roll
GSP(SEC2)	Host/PCIe staging buffer	FMC	eHSM	强制签名 + 加密 + roll
PM/RAS/Codec	Host/PCIe staging buffer	GSP	eHSM	USER/PROD 默认签名
FMC recovery package	Host/BMC -> OOB MCU	OOB MCU 预检查；最终 BootROM/eHSM 裁决	OOB MCU 授权检查 + BootROM/eHSM verify/decrypt	包授权 + FMC 自身签

3.3 失败处理

- FMC verify/decrypt 失败：BootROM 停止跳转 FMC，可选记录 `boot_fail_reason`、`ehsm_status`、`manifest_status` 等诊断信息，等待复位或下一次刷机后重启。
- FMC manifest policy 失败：按 FMC 验证失败处理，不跳转。
- GSP 验证失败：FMC 拒绝跳转 GSP，保持基础安全控制面，允许 Host 通过 PCIe 重新下发 GSP。
- Runtime 固件验证失败：GSP 拒绝对应微核 release，Host 可重新下发对应 Runtime 包。
- eHSM 未 ready 或 mailbox 超时：BootROM/FMC/GSP 进入受控失败路径，不允许旁路。
- Host 发现卡连接超时、PCIe 枚举失败或心跳超时：Host/BMC 触发 OOB MCU 刷机流程；该触发不依赖 BootROM 已经写入 `recovery_required`。
- USER/PROD 生命周期不允许自动降级到非安全启动。

3.4 BootRom 加载 FMC 流程



步骤说明：

1. Reset 后 BootROM 做最小初始化并等待 eHSM。
2. eHSM ready 后，BootROM 从固定 NOR Flash 偏移读取 FMC。

- 3. eHSM verify/decrypt 通过后进入 manifest 检查。
- 4. manifest、load/entry 通过后加载并跳转 FMC。
- 5. 任一失败进入 Halt，等待 OOB 刷机或电源复位后重新启动。

3.5 OOB MCU 刷写约束

项目	约束
OOB MCU 自身可信	OOB MCU 必须有独立 secure boot、升级鉴权、调试关闭、生命周期或板级
触发条件	Host PCIe 枚举失败、连接超时、心跳超时、运维/RMA/制造授权刷机；不 recovery_required。
QSPI ownership	OOB MCU 执行刷写前必须获得 NOR Flash QSPI ownership，SoC 侧不得
写入范围	OOB MCU 只允许写固定 FMC 分区和授权允许的恢复状态区
包授权	FMC recovery package 必须有 update authorization token 或等价签名授
写后校验	OOB MCU 必须执行 readback/hash 校验；该校验只证明写入一致性，不证
最终验证	BootROM/eHSM 必须在下一次启动时重新验证签名、解密、rollback、ma entry 白名单。
重发重刷	若掉电或写入失败，Host/BMC 可重新下发 package，OOB MCU 可重复擦

4. 固件包格式与验证解密流程

4.1 eHSM native package

FMC、GSP 和受保护 Runtime 固件采用 eHSM native secure image package。物理格式以 eHSM TRM 为准，NGU 不再定义第二套 physical verification header。



- 1. eHSM native header 承载 eHSM 原生认证/加密控制字段。
- 2. Code region 由 eHSM verify/decrypt 保护。
- 3. NGU protected manifest 位于 Code region 内，eHSM PASS 后才可被 BootROM/SEC 解析。
- 4. payload 也位于受保护 Code region 内，manifest 指示其类型、版本、load/entry 和 measurement slot。

4.2 eHSM header 字段使用规则

字段	NGU 使用规则
Signature	eHSM 原生签名或认证材料，NGU 不另行定义尾随 signature。
Public_Key	eHSM 原生公钥，不作为 NGU 证书区。
Encrypt_IV	加密 IV/nonce 元数据，语义跟随 eHSM profile。
Valid_Flag	eHSM image valid marker，（magic value 0x8E97645D）
Image_Type	只表达 eHSM 原生类型（SoC / eHSM），不承载 NGU 的 FMC/GSP/PM/RAS/Codec。
Plain_Flag	必须与 NGU manifest 的 decrypt_required 一致；FMC/GSP 正式路径不得为明文 profile。
Naked_Flag	仅用于 eHSM 允许的测试/开发 profile，USER 不依赖裸镜像。(是否是裸镜像)
Code_Size	覆盖 manifest、payload、padding 的完整 Code region。
Version_Counter	eHSM physical anti-rollback 输入；NGU rollback_domain 只能映射到该机制。
Reserved	必须按 TRM 处理，不承载 NGU 自定义字段。

4.3 NGU800 protected manifest

NGU800 项目级字段全部放入 eHSM 认证/解密后的 Code region。BootROM/FMC/GSP 只能在 eHSM PASS 后解析 manifest。

字段语义	用途
magic / manifest_version	标识 manifest ABI，拒绝未知或不兼容版本。
image_type	表达 FMC、GSP、PM、RAS、Codec 等项目级类型。
security_policy_flags	表达 sign_required、decrypt_required、rollback_required、measure_required、release_policy。
payload_offset	定位 payload，必须完全落入 Code_Size。
load_addr / entry_addr	装载和跳转地址。
version	项目级版本视图，映射到 eHSM version counter。
measurement_slot	指定进入 measurement table 的 slot。
lifecycle_mask	指定允许的 lifecycle 集合，最终以 eHSM/eFuse lifecycle 为准。
board_binding_policy	板级绑定结果的 attestation/release 表达。
expected_algorithm_profile	与 eHSM control field 做一致性检查，不覆盖 eHSM 算法选择。
payload_digest	payload digest 描述，hash profile 跟随产品策略。

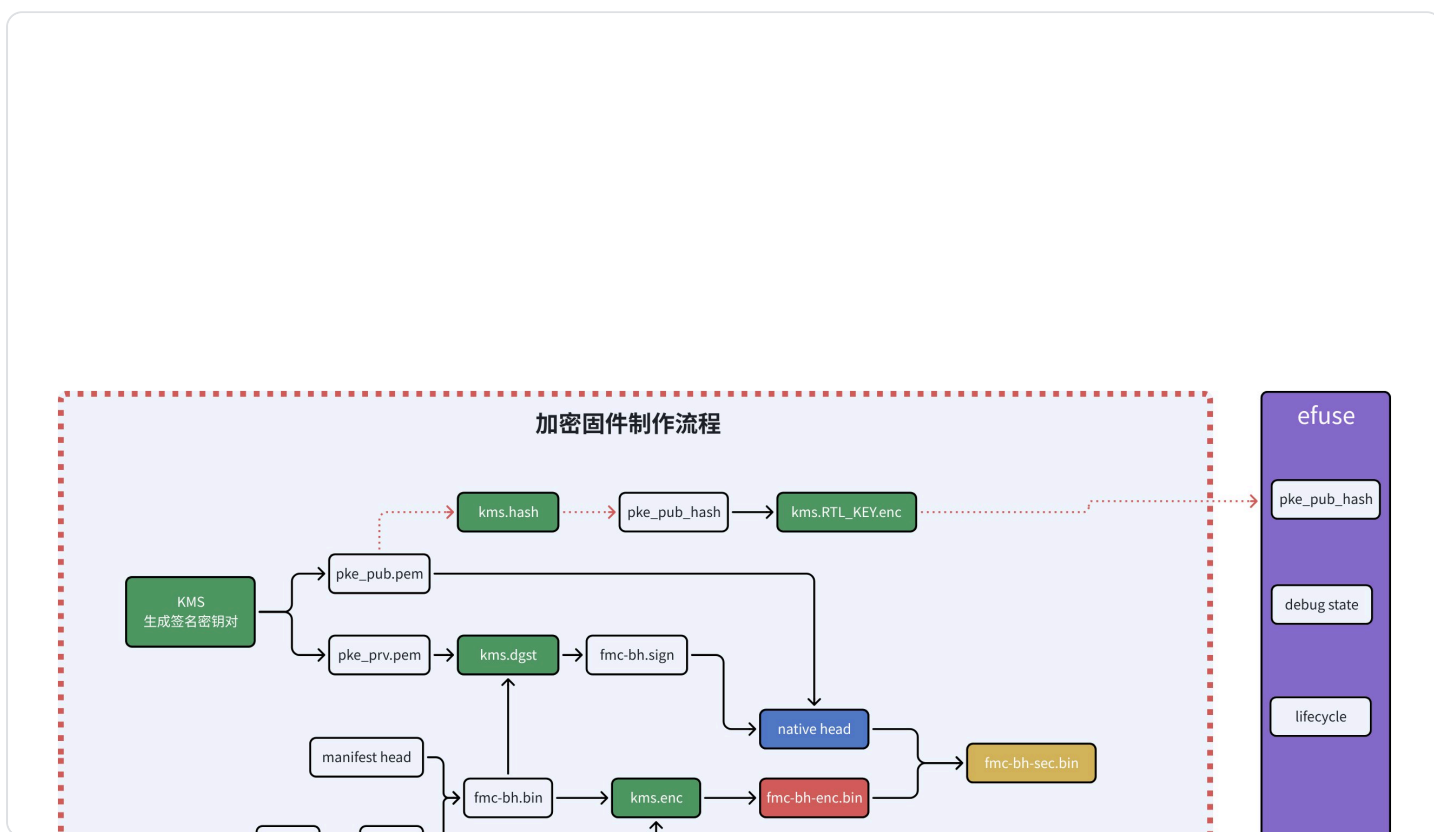
4.4 设备侧验证解密流程

1. BootROM/FMC/GSP 定位 package，检查 `image_addr`、`image_len`、`output_addr` 是否处于白名单区域。
2. 调用 eHSM `bl_verify_image`、`soc_verify` 或项目 wrapper。

3. eHSM 解析 native header, 检查 Image_Type、Plain_Flag、Code_Size、Version_Counter、key/signer/revoke/rollback/signature。
4. eHSM 对 Code region 进行认证和解密, 输出 plaintext Code region 到受控 output buffer。
5. 调用方在 eHSM PASS 后解析 NGU manifest。
6. 调用方检查 ngu_image_type、policy flags、lifecycle、rollback、load/entry、measurement slot、algorithm profile。
7. 通过后记录 measurement, 装载 payload, 并执行 release 或 jump。

4.5 固件包制作与设备侧校验流程图

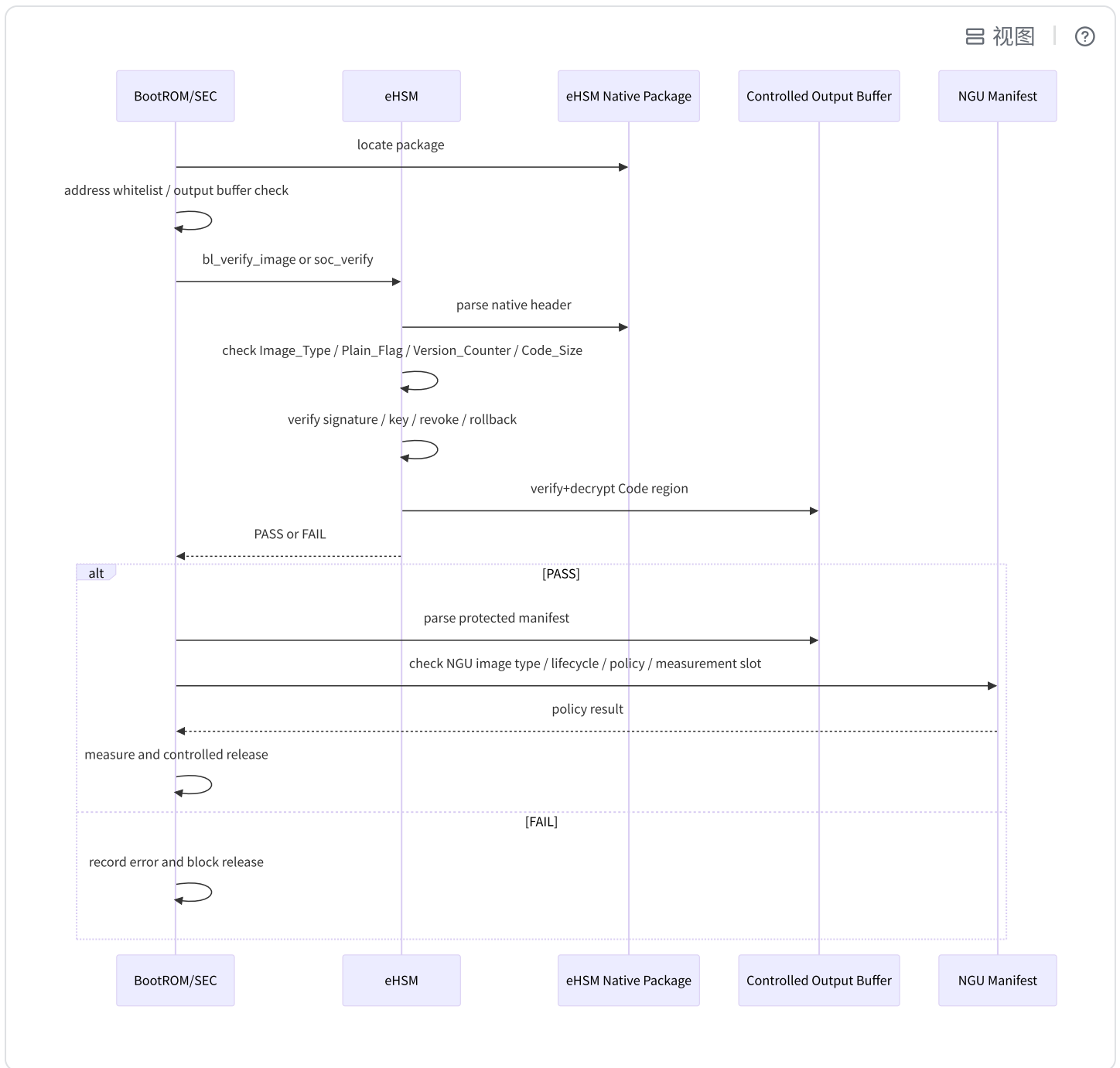
4.5.1 FMC 安全固件制作流程



1. 编译生成 fmc.bin, 并准备好 manifest, 生成 fmc-bh.bin;
2. 由 KMS 生成签名密钥 sign_pub_key, sign_prv_key;
3. 对 sign_pub_key 计算 pubkey_hash, 并由 RTL_KEY 进行加密, 在制造阶段灌装至 eFuse;
4. 计算 fmc-bh.bin 的 hash 值 (fmc-bh-hash);
5. 使用 sign_prv_key 对 fmc-bh-hash 签名, 生成 signature, 填入 native haed 的对应字段;
6. 由 KMS 提供对称密钥 FW_Enc_Key, 对fmc-bh.bin 加密生成 fmc-bh-enc.bin;
7. 组装 native header, 并与fmc-bh-enc.bin 拼装成最终 fmc-bh-sec.bin(fmc 的安全固件);
8. 对称密钥 FW_Enc_Key 经过 RTL_KEY 加密后, 在制造阶段灌装至eFuse。

4.5.2 FMC 固件验证流程

设备侧 verify/decrypt 与 manifest policy 流程



1. BootROM 首先定位 eHSM native package，并检查 image_addr、output buffer、load buffer 等地址是否落在白名单范围内。
2. BootROM 通过 bl_verify_image 或 soc_verify 调用 eHSM，并传入 package 地址、输出 buffer 地址和期望 profile。
3. eHSM 解析 native header，检查 Image_Type、Plain_Flag、Version_Counter、Code_Size 等 native 字段。
4. eHSM 完成 key、signer、revoke、rollback、signature 检查，并按 eHSM profile 对受保护 Code region 执行 verify + decrypt。

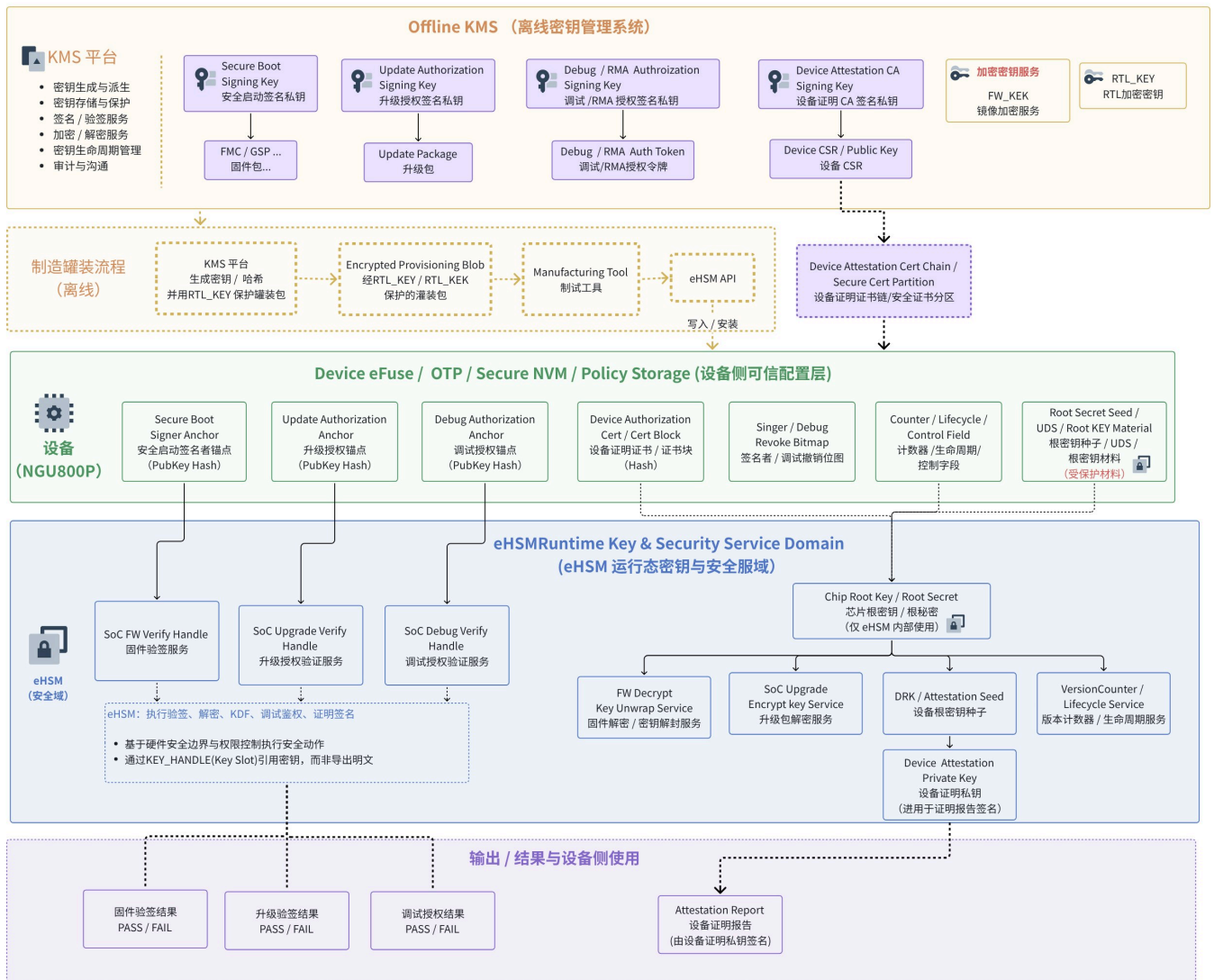
5. eHSM 将解密后的明文 Code region 输出到 Controlled Output Buffer，并向 BootROM/SEC 返回 PASS 或 FAIL。
6. 只有 eHSM PASS 后，BootROM 才解析 NGU Manifest；eHSM FAIL 时必须记录错误并阻断 release。
7. BootROM 根据 NGU Manifest 检查 ngu_image_type、security policy、lifecycle、rollback domain、measurement slot、load_addr、entry_addr 等项目级策略。
8. manifest policy 通过后，BootROM 记录 measurement，并执行 load + jump 或 release。
9. manifest policy 失败、地址越界、entry 不合法、rollback 不满足、decrypt 未执行或结果失败时，均不得继续启动或释放目标核。

5. 密钥、证书与制造灌装

本章定义 NGU800P 安全方案中的 Root of Trust、密钥体系、证书体系、eFuse 持久化对象、eHSM 运行态密钥使用方式，以及制造阶段的灌装流程

5.1 密钥证书体系

Root of Trust、密钥与证书体系架构图



1. Offline KMS 是外部信任源，负责保存签名私钥、授权私钥、设备证明 CA 私钥，并提供固件加密、CEK 服务和制造阶段 provisioning blob 生成能力。
2. 外部签名私钥不进入设备。设备侧保存的是 signer anchor、authorization anchor、debug anchor、revoke bitmap、key epoch、lifecycle、counter、control field、root material 和证书信息。
3. 需要写入 eFuse key slot 的 key/hash，由 KMS 生成后使用 RTL_KEY 加密成 provisioning blob，再由制造工站调用 eHSM `install_encrypt_key` 写入设备。
4. eFuse / OTP / Secure NVM / Policy Storage 是设备侧可信配置层，负责持久保存“设备信谁、允许什么策略、当前处于什么安全状态”。
5. eHSM 是运行态安全服务执行域。eHSM 上电后加载、解密或映射 eFuse 中的对象，形成内部 `KEY_HANDLE`、key slot 和安全上下文。
6. eHSM 对外提供固件验签、固件解密、升级授权验证、Debug/RMA 鉴权、计数器/生命周期服务和证明报告签名服务。

- 7. Device Attestation Private Key 只用于签署证明报告，不用于固件验签、固件解密、升级授权或 Debug/RMA 授权。
- 8. 完整设备证书链存放在安全证书分区，eFuse/OTP 中仅保存证书 hash、issuer anchor 或 cert slot id。
- 9. OOB MCU 不访问 SoC Root Key、不访问 eHSM 内部密钥，也不替代 eHSM 判断刷入的 FMC 是否可信。

体系	私钥位置	设备侧保存内容	主要用途
外部签名/授权体系	离线 KMS 服务器	signer anchor、public key hash、revoke bitmap、debug anchor、update anchor	固件签名、升级授权 token、设备证书签
设备内部 eHSM key 体系	eHSM/eFuse/KMU 内部	key handle、control field、counter、lifecycle、key slot policy	固件解密、key unw private key 使用、c 制。
设备身份与证书体系	证明私钥在 eHSM；证书签发私钥在离线 HSM	Device Attestation Cert、issuer anchor、cert serial/hash	设备身份和 report 签

5.2 OTP Key 安装 API 策略

制造阶段写入 eFuse 的密钥类对象，主要使用两类 eHSM OTP Key 安装 API：

代码块

```
1  install_random_key
2  install_encrypt_key
```

这两类接口只用于持久 OTP Key Slot 类对象，不用于完整证书链、生命周期、控制字段或普通运行态 session key。

5.2.1 install_random_key

install_random_key 用于由 eHSM 内部生成随机密钥，并使用对应 Root Key 加密后写入 eFuse。

- 适用对象：

对象	说明
设备内部随机对称密钥	不希望密钥在 KMS/HSM 外部明文出现
Device Attestation Private Key	推荐设备内生成私钥，私钥不出设备
device-local secret	设备本地保护或派生使用
特定量产随机密钥	由 eHSM TRNG 生成并写入 OTP

- 不适用对象：

对象	原因
Secure Boot Signer Anchor	需要安装外部签名公钥 hash，不能随机生成
Update Authorization Anchor	需要安装外部升级授权公钥 hash
Debug Authorization Anchor	需要安装外部 Debug/RMA 授权公钥 hash
cert hash / issuer anchor	是外部证书系统确定的摘要或锚点

5.2.2 install_encrypt_key

install_encrypt_key 用于安装外部 KMS/HSM 指定的密钥、私钥或公钥 hash。

制造阶段流程如下：

代码块

```
1  KMS/HSM 生成或派生 key/hash
2      -> 计算 CRC
3      -> 组装 key/hash + CRC + padding
4      -> 使用 RTL_KEY / RTL_KEK 加密
5      -> 形成 encrypted provisioning blob
6      -> 制造工站调用 eHSM install_encrypt_key
7      -> eHSM 内部使用 RTL_KEY / RTL_KEK 解密
8      -> eHSM 校验 key_type / key_slot_id / CRC / lifecycle 权限
9      -> eHSM 使用 Chip Root Key 或 Device Root Key 重新加密
10     -> 写入 OTP/eFuse Key Slot
```

• 适用对象：

对象	推荐原因
SOC Encrypt Key	固件 Code 区解密密钥，由 KMS/HSM 生成或派生
SOC Upgrade Encrypt Key	升级包外层解密密钥
SOC FW Verify Key / PubKey Hash	安全启动 signer 公钥 hash
SOC Upgrade Verify Key / PubKey Hash	升级授权 signer 公钥 hash
SOC Debug Verify Key / PubKey Hash	Debug/RMA 授权 signer 公钥 hash
User Auth Key	USER 模式受控操作鉴权 key

5.3 设备证明密钥与证书策略

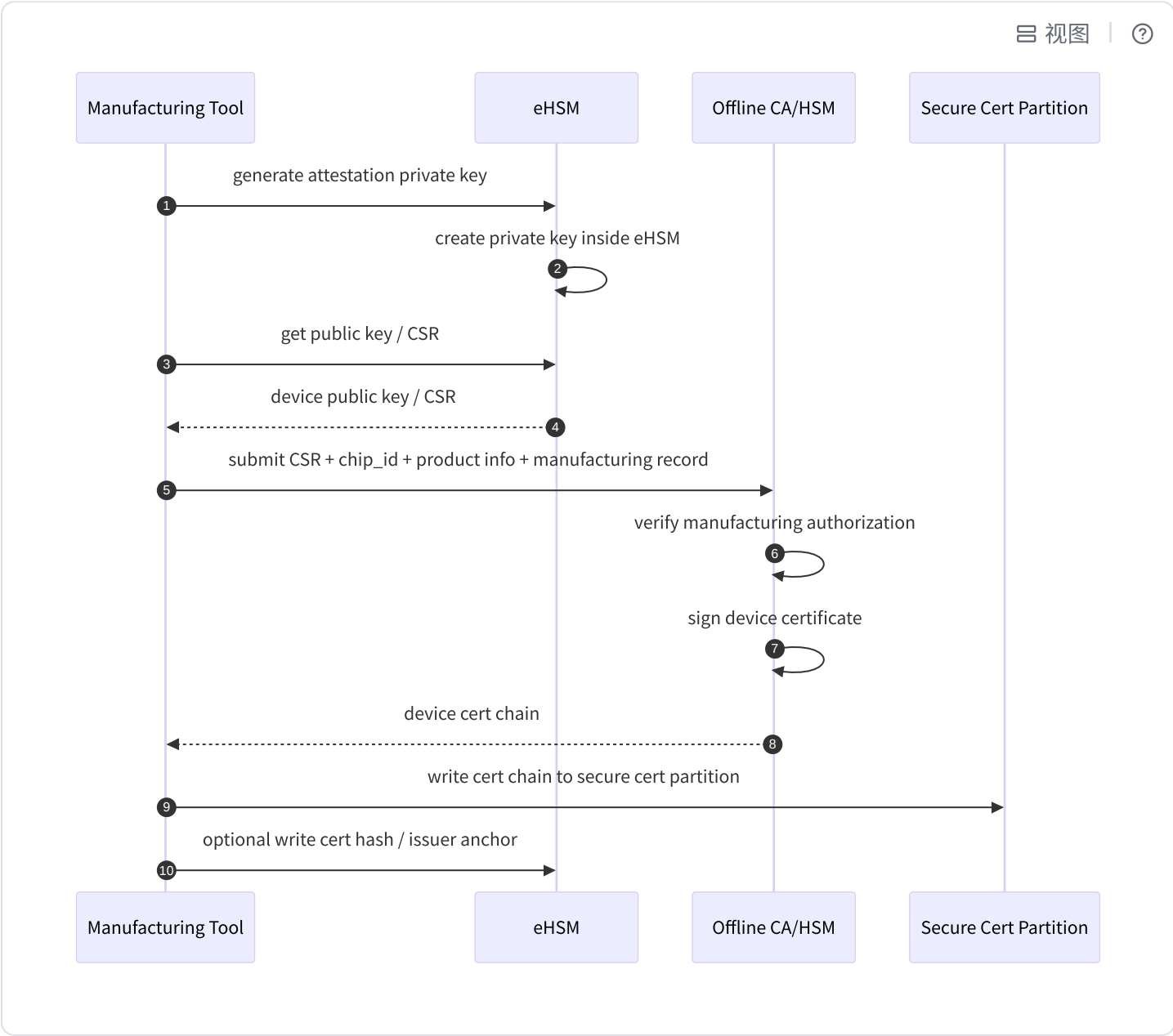
5.3.1 设备证明对象关系

对象	位置	说明
Device Attestation Private Key	eHSM 内部 key slot / secure storage	用于签署 attestation report
Device Attestation Public Key	CSR / 证书中	可公开
Device Attestation Cert	Secure cert partition	证明设备公钥属于合法设备
Device Attestation CA Signing Key	Offline CA/HSM	签发设备证书，不进入设备
Root CA Cert	Host/Verifier 预置	验证端信任锚

5.3.2 证书生成策略

设备证明私钥由 eHSM 内部生成，私钥不出设备。制造阶段仅导出公钥或 CSR，由离线 CA/HSM 签发设备证明证书。

Device Attestation Key / CSR / 证书签发流程图



步骤说明：

1. 制造工具请求 eHSM 内部生成设备证明私钥。
2. 私钥在 eHSM 内部生成和保存，不出设备。
3. eHSM 输出设备公钥或 CSR。
4. 制造工具将 CSR、chip_id、product info 和制造记录提交给离线 CA/HSM。
5. CA/HSM 校验制造授权后签发设备证书。
6. 制造工具将设备证书链写入安全证书分区。
7. eFuse 中可选保存证书 hash、issuer anchor 或 cert slot id。
8. 运行时 eHSM 使用设备证明私钥签署 attestation report。

5.4 证书链与证书存储策略

5.4.1 总体策略

NGU800 第一版采用**静态 X.509 设备证明证书链**方案。设备侧不采用 DICE 动态证书链，不在每一级启动过程中动态生成证书。为便于后续对齐 SPDM / 业界设备证明接口，设备侧 NOR Flash 中保存完整 X.509 certificate chain。典型结构为：

代码块

```
1  Root CA Cert
2      -> Intermediate CA Cert
3      -> Device Attestation Cert
```

其中：

代码块

```
1  Root CA Cert:
2      根 CA 证书，对应整条证明证书链的最高信任根。
3      Verifier 最终是否信任该 Root CA，以 Verifier 本地 trust store 为准。
4
5  Intermediate CA Cert:
6      中间 CA 证书，用于签发 Device Attestation Cert。
7      可按产品线、客户、批次或设备类型划分。
8
9  Device Attestation Cert:
10     设备证明证书，包含 Device Attestation Public Key、device_uuid、
11     chip_id、product_id 等设备身份信息。
```

NGU800 设备侧保存 Intermediate CA Cert 和 Device Attestation Cert。

5.4.2 完整证书链存储位置

完整 Device Attestation Cert Chain 存放在 NOR Flash Secure Cert Partition，该分区用于保存设备证明证书链，不与普通固件、配置区、日志区混用。

证书链本身属于公开身份材料，主要用于让外部 Verifier 获取 Device Attestation Public Key 并验证 Attestation Report 签名，因此不需要加密保存。但是，该分区的写入必须受控，不能允许 Host 或普通运行时软件直接裸写。**更新或者烧写方式参考固件的更新和刷机流程。**

5.4.3 证书链存储结构

定义如下证书 slot header：

代码块

```
1  typedef struct {
2      uint32_t magic;
3      uint16_t version;
4      uint16_t header_len;
5
6      uint32_t cert_slot_id;
7      uint32_t cert_format;          /* 1: X509_DER_CHAIN */
8      uint32_t cert_chain_len;
9      uint32_t cert_chain_offset;
10
11     uint8_t  cert_chain_hash[32];
12
13     uint32_t cert_epoch;            /* 可选，用于证书链更新代际 */
14     uint32_t flags;
15
16     uint32_t reserved[8];
17
18     /*
19      * followed by:
20      *   X.509 DER Certificate Chain Blob
21      */
22 } ngu_cert_slot_header_t;
```

证书链 blob 可以采用 length-value 方式组织：

代码块

```
1  typedef struct {
2      uint32_t cert_count;
3
4      /*
5      * followed by:
```

```

6      *   uint32_t cert0_len;
7      *   uint8_t  cert0_der[cert0_len];
8      *
9      *   uint32_t cert1_len;
10     *   uint8_t  cert1_der[cert1_len];
11     *
12     *   ...
13     */
14 } ngu_x509_cert_chain_blob_t;

```

字段说明：

代码块

```

1  cert_slot_id:
2      证书槽编号。
3
4  cert_format:
5      固定为 X509_DER_CHAIN。
6
7  cert_chain_len:
8      整个证书链 blob 长度。
9
10 cert_chain_hash:
11     Hash(X.509 DER Certificate Chain Blob)，用于写入后校验和分区完整性检查。
12
13 cert_epoch:
14     证书链代际，可选。后续用于证书链更新和防回滚。
15
16 flags:
17     证书槽状态，例如 valid、staging、active、recovery 等。

```

5.4.4 证书链 slot 策略

为对齐 SPDM certificate slot 思路，Secure Cert Partition 支持多证书槽。

如果资源有限，可以只实现：

代码块

```

1  cert_slot_0: current certificate chain
2  cert_slot_1: staging / next certificate chain

```

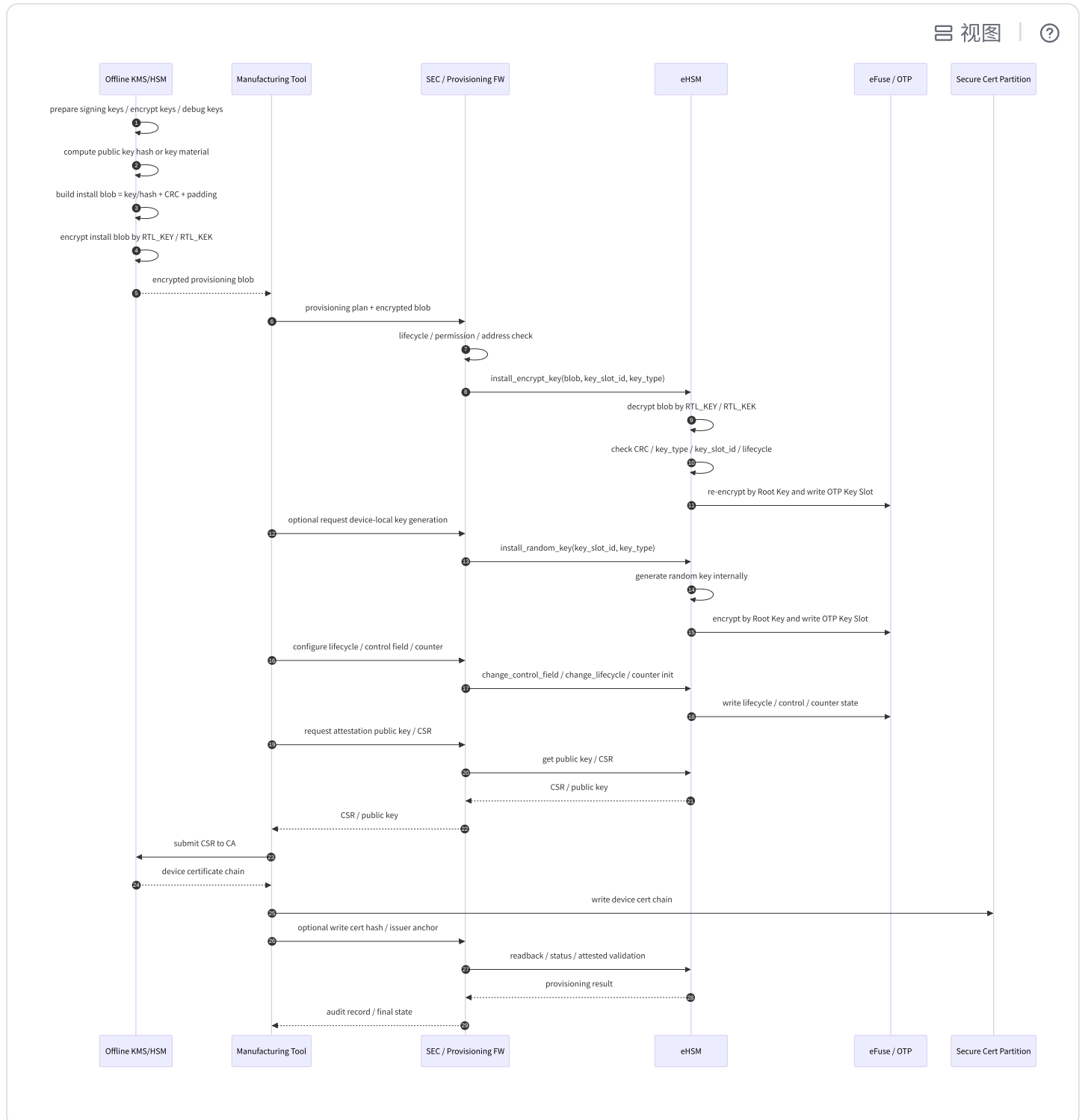
每个 slot 保存：

- 1 Cert Slot Header
- 2 +
- 3 X.509 DER Certificate Chain Blob

slot 切换不能由 Host 直接修改 Flash 元数据完成，必须通过受控更新流程完成。

5.5 制造灌装总流程

制造灌装总流程图



1. KMS 准备启动、升级、Debug、加密相关密钥。
2. 对于非对称验签 key，KMS 计算 signer public key hash。
3. 对于对称加密 key，KMS 生成或派生 SOC Encrypt Key、SOC Upgrade Encrypt Key 等。
4. KMS 构造 key/hash + CRC + padding 的安装 blob。
5. KMS 使用 RTL_KEY 对安装 blob 加密。
6. 制造工具将 encrypted provisioning blob 下发给 SEC / Provisioning FW。
7. SEC 检查 lifecycle、权限和参数边界后调用 eHSM `install_encrypt_key`。
8. eHSM 使用 RTL_KEY 解密安装 blob，并校验 CRC、key type、slot id 和 lifecycle 权限。
9. eHSM 使用 Chip Root Key 或 Device Root Key 重新加密，并写入 eFuse key slot。
10. 对于设备内生成的私钥或随机 key，制造工具调用 `install_random_key`。
11. 生命周期、控制字段和版本计数器不走密钥安装 API，而通过独立控制接口配置。
12. 设备证明证书链写入安全证书分区。
13. 灌装完成后执行 readback/status/attested validation，并生成制造审计记录。

5.6 安全约束

1. RTL_KEY 只用于制造阶段密钥安装 blob 的传输保护，不作为运行态业务密钥使用。
2. `install_encrypt_key` 的输入 blob 不得在工站本地明文落盘。
3. 进入 USER 生命周期前，应关闭 TEST/DEV/MANU 模式下的任意 OTP 写能力。
4. USER 模式下不得允许重新安装启动验签 key、固件解密 key、升级授权 key。
5. Debug/RMA 授权 key 应与启动验签 key、升级授权 key 分离。
6. Device Attestation Private Key 不得导出，不得用于固件验签、固件解密、升级授权或 Debug/RMA 授权。
7. 完整证书链可被读取，但证书链所在分区应具备防篡改机制；必要时使用 eFuse 中的 cert hash 进行绑定。
8. OOB MCU 不得访问 SoC eHSM Root Key、SOC Encrypt Key、SOC FW Verify Key、Device Attestation Private Key 等密钥材料。
9. 正常更新写 Flash 前必须完成 eHSM 验证。
10. OOB MCU 刷机完成后的镜像，仍必须在下一次 SoC 启动时由 BootROM + eHSM 验证通过后才能执行。
11. 所有 key install、lifecycle change、control field change、debug auth、firmware update 和 OOB recovery 事件都应进入审计记录。

12. USER 前必须清理测试 key、测试证书、测试 signer anchor 和开发调试白名单。

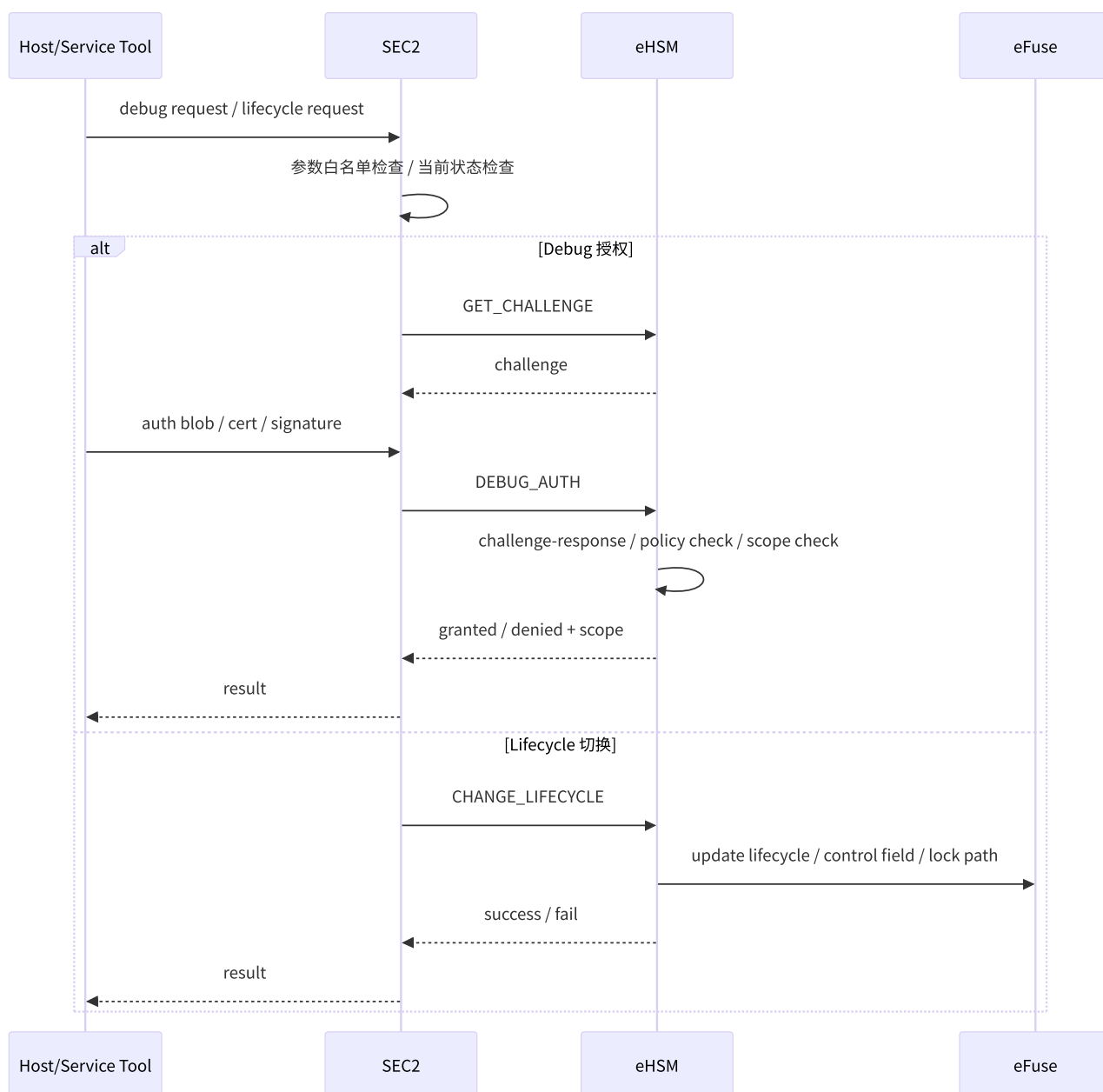
6. 生命周期与安全 Debug

6.1 生命周期模型

状态	语义	启动策略	Debug 策略
TEST	裸片/封测/初测	可开放测试路径，不等价量产	开发授权
DEV	开发/EVB	可安全/非安全启动	开发授权
MANUFACTURE	正式制造	优先安全启动	有限受控
USER	量产交付	强制安全启动	默认关闭
DEBUG/RMA	受权返修	受控签名启动	授权后有限开放

6.2 Debug 授权流程

Debug 授权与生命周期切换时序



1. Host/BMC/服务工具向 GSP 发起 debug request，携带目标、scope 和用途。
2. GSP 检查 lifecycle、目标白名单、当前 secure boot 状态和请求格式。
3. GSP 调用 eHSM `GET_CHALLENGE`。
4. 工具提交 Debug/RMA scoped token，对 challenge、UID、scope、expiry、reason 签名。
5. GSP 调用 eHSM `DEBUG_AUTH`。
6. eHSM 校验 token、证书/anchor、revoke、lifecycle、scope 和过期策略。
7. 通过后返回 granted scope、expiry policy、audit id。
8. GSP 只打开授权范围内的 JTAG/trace/debug window，并设置自动关闭条件。
9. 超时、复位、生命周期切换、安全错误或显式 `CLOSE_DEBUG` 时关闭 scope 并记录审计。

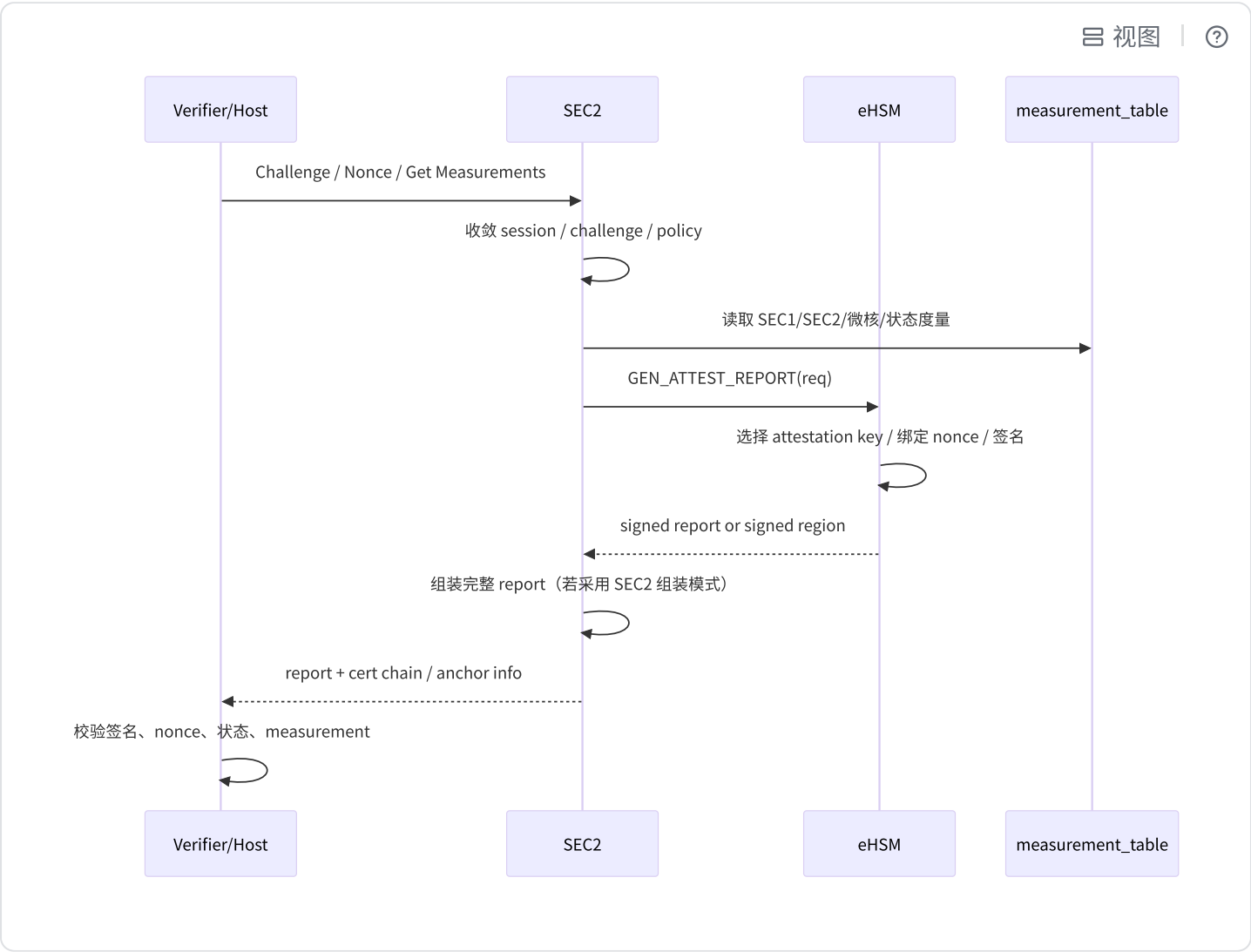
6.3 Debug scope

是否要划分Debug scope, tbd

如 CPU halt / single-step、Secure memory、GPU register / DRAM access、Flash access等

7. 设备远程认证

认证报告生成时序图



- 1. Verifier 发起 challenge/nonce。
- 2. SEC2 汇总 measurement_table、lifecycle、debug 和 secure boot 状态。
- 3. SEC2 请求 eHSM 签名报告或签名区域。
- 4. eHSM 选择证明密钥并绑定 nonce/session。
- 5. SEC2 返回 report、证书链或 anchor 信息给 verifier。

7.1 角色

角色	位置	职责
Requester	Host driver / Host security agent	发起 SPDM、获取证书、challenge、measurements、report
Responder	GSP(SEC2)	SPDM Responder 状态机、measurement 汇总、report 组织
Signing service	eHSM	使用 Device Attestation Private Key 对 report 或 signed reg
Verifier	Host 本地策略模块或远端验证服务	校证书链、签名、nonce、measurement、lifecycle、deb 略。

7.2 SPDM 流程

1. 版本、能力和算法协商： GET_VERSION 、 GET_CAPABILITIES 、 NEGOTIATE_ALGORITHMS 。
2. 获取证书： GET_DIGESTS 、 GET_CERTIFICATE 。
3. 挑战认证： Host 发起 CHALLENGE ，设备使用证明私钥签署 challenge 上下文。
4. 会话建立：需要加密会话时执行 KEY_EXCHANGE 、 FINISH 。
5. 获取测量： Host 发起 GET_MEASUREMENTS ， GSP 从 measurement table 读取度量和状态，调用 eHSM 签名后返回 report。
6. 策略判定： Verifier 校证书、签名、nonce、版本、状态、measurement 和 rollback 门限。

国密路径使用 SM3 作为 hash、SM2 作为 signature/cert key，安全会话可采用 SM4-GCM。国际路径保留 SHA-256/SHA-384、ECDSA-P256/P384、RSA3072 等 profile。

7.3 Measurement table

measurement table 至少覆盖：

- BootROM / immutable identity / ROM version。
- FMC(SEC1) hash、version、rollback counter、image protection policy、decrypt_applied。
- GSP(SEC2) hash、version、rollback counter、image protection policy、decrypt_applied。
- PM/RAS/Codec 等 Runtime 固件 hash、version、rollback counter。
- lifecycle state、debug state、secure boot enable、anti-rollback enable。
- chip_id / device_uuid / die info。

7.4 Report 结构

设备输出的证明报告为一份结构化数据对象。报告包含以下几个部分。

- report header
- measurement集合
- cert chain

report header

measurement

cert chain

signature

7.4.1 Report Header

代码块

```
1  typedef struct {
2      uint32_t magic;
3      uint16_t report_version;
4      uint32_t header_len;
5      uint32_t total_len;
6
7      uint8_t  report_uuid[16];
8      uint8_t  device_uuid[16];
9      uint8_t  chip_id[16];
10
11     uint8_t  requester_nonce[32];
12     uint32_t session_id;
13
14     uint32_t lifecycle_state;
15     uint32_t debug_state;
16     uint32_t debug_scope;
17     uint32_t secure_boot_state;
18     uint32_t update_state;
19     uint32_t anti_rollback_state;
20
21     uint32_t hash_algo;
22     uint32_t sig_algo;
23     uint32_t cert_format;
24     uint32_t cert_slot_id;
25
26     uint32_t block_count;
27     uint32_t signed_region_offset;
28     uint32_t signed_region_len;
29 } ngu_attest_report_header_t;
```

7.4.2 Measurement Block

Measurement Block 是认证报告的核心，用于承载安全启动和运行态的可信对象。

1) Measurement Block Header

代码块

```

1  typedef struct {
2      uint16_t block_type;
3      uint16_t block_version;
4      uint32_t block_len;
5
6      uint32_t fw_count;
7      uint32_t flags;
8      uint32_t reserved0;
9  } ngu_measurement_block_header;

```

`block_type`：固定为 measurement block

`block_version`：当前版本

`block_len`：整个 measurement block 总长度

`flags`：可选，标识这个 block 的整体属性

其他预留

2) fw info

代码块

```

1  typedef struct {
2      uint32_t fw_type;
3      uint32_t hash_len;
4      uint8_t  hash[64];
5      uint32_t fw_version;
6      uint32_t rollback_counter;
7      uint32_t reserved0;
8  } ngu_meas_fw_info;

```

3) Measurement Block

代码块

```

1  typedef struct {
2      ngu_measurement_block_header hdr;
3      /*
4       * followed by hdr.fw_count:
5       *   ngu_meas_fw_info  fw_info[];
6       */
7      uint8_t fw_info[];
8  } ngu_measurement_block

```

签名覆盖范围必须包含 header、identity、nonce/session、measurement、lifecycle/debug/secure boot/rollback、firmware version 和 cert chain metadata。Verifier 不能只校验签名，还必须校验状态和策略。

8. 密钥轮换管理与鉴权策略

本章单独定义 NGU800 的密钥轮换对象、槽位预留、状态机、正常更新场景下的轮换流程、OOB 刷机恢复场景下的轮换流程，以及单 FMC 固定分区下的撤销策略。其目标为了让设备在 **密钥泄露、算法升级、客户接管、生命周期变化、证书过期/吊销** 时仍然可控、可恢复、可继续安全运行

8.1 需要支持轮换的密钥与锚点

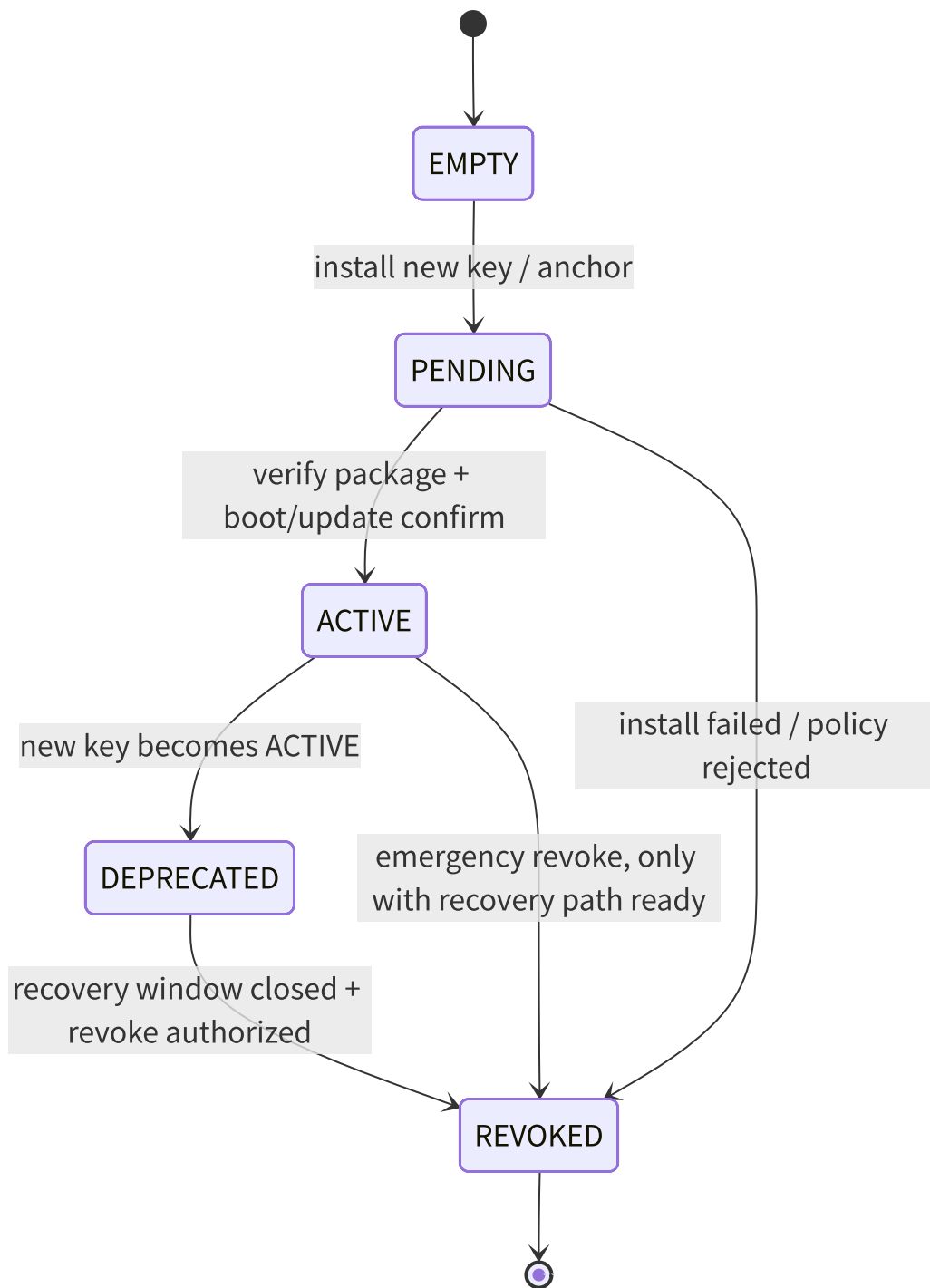
NGU800 中需要预留轮换能力的对象，主要是外部授权体系对应的 trust anchor、public key hash 以及支撑密钥轮换的 key epoch、revoke policy 等控制信息，而不是所有设备内部密钥。

对象	是否需要轮换	槽位	管理主体	用途
SoC FW Verify Anchor	是	2~4	eHSM / eFuse	验证 FMC、GSP、Runtime 固件签名。
update_authorization_anchor	是	2~4	eHSM / eFuse	验证正常升级包、key rotation capsule、FMC update package。
fw_encrypt_key	是	2~4	eHSM key policy / counter	控制固件包加密、CEK unwrap 或镜像解密代际。
debug_authorization_anchor	是	2~4	eHSM / eFuse	验证 Debug/RMA scoped token。
device_attestation_cert_chain	是，证书链可切换	2~4	Flash 安全区	支持证书续期、CA rollover、客户接入。

8.2 密钥状态机

所有可轮换的授权锚和 key epoch 采用统一状态机。

密钥槽位状态机图



图下步骤说明：

1. **EMPTY** 表示槽位未使用。
2. 新 key 或 anchor 写入后先进入 **PENDING**，不得立即替代当前 **ACTIVE**。
3. 只有新包通过验证、启动或更新确认完成后，**PENDING** 才能变为 **ACTIVE**。
4. 旧 **ACTIVE** 不直接撤销，先进入 **DEPRECATED**，用于过渡期恢复和兼容旧包。
5. 确认 OOB recovery 包、rollback counter、attestation 状态均切到新策略后，旧 key 才能进入 **REVOKED**。

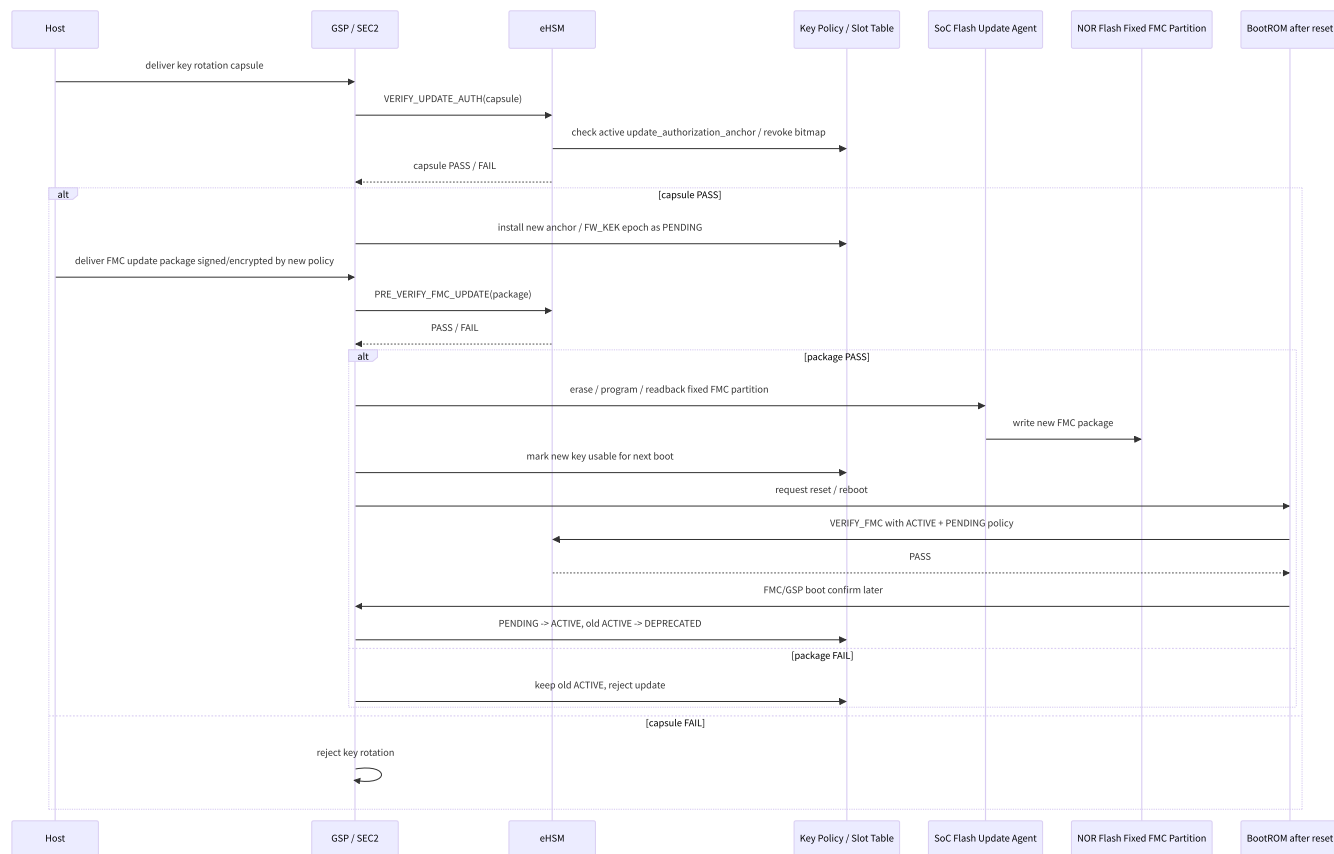
状态语义如下：

状态	语义	是否允许验签	是否允许签发新包	是否允许撤销
EMPTY	未安装	否	否	不适用
PENDING	已安装，等待验证/确认	受限允许，用于新包预验证或一次启动验证	否	可回滚为 REVOKED
ACTIVE	当前主用 key	是	是	需要授权 capsule
DEPRECATED	过渡期兼容旧包或恢复包	受限允许	否	满足条件后可撤销
REVOKED	已撤销	否	否	不可恢复为有效态
LOCKED	可选锁定态	取决于策略	否	取决于生命周期

8.3 正常固件更新中的密钥轮换流程

正常固件更新路径运行在 SoC 已经正常启动之后，Host 通过 PCIe 将 key rotation capsule 或 firmware update package 下发给 GSP，GSP 调 eHSM 完成鉴权和策略更新。

图 7-2 正常更新路径下的 key rotation 时序



图下步骤说明：

1. Host 通过 PCIe 下发 key rotation capsule。
2. GSP 调 eHSM 使用当前 update authorization anchor 验证 capsule。
3. 验证通过后，新 boot signer、update anchor 或 FW_KEK epoch 先安装为 **PENDING**。
4. Host 下发使用新策略签名/加密的 FMC update package。
5. GSP 调 eHSM 预验证新 FMC 包，通过后才允许 SoC Flash Update Agent 写固定 FMC 分区。
6. 重启后，BootROM/eHSM 允许 **ACTIVE + PENDING** 集合验证 FMC。
7. 新 FMC/GSP 启动确认后，新 key 从 **PENDING** 切到 **ACTIVE**，旧 key 进入 **DEPRECATED**。
8. 如果 capsule 或 package 验证失败，旧 **ACTIVE** 保持不变，不执行 revoke。

9. 板级/OOB 安全边界

本章将 OOB MCU / 板级安全 MCU 纳入板级安全边界。BMC/OOB Host 仍按外部不可信请求方处理；OOB MCU 是板级安全控制点，可以承载带外管理、电源/复位、JTAG MUX 控制和 QSPI 刷写 FMC，但不替代 eHSM Root of Trust，不持有根密钥，不直接放行任何 SoC 固件。

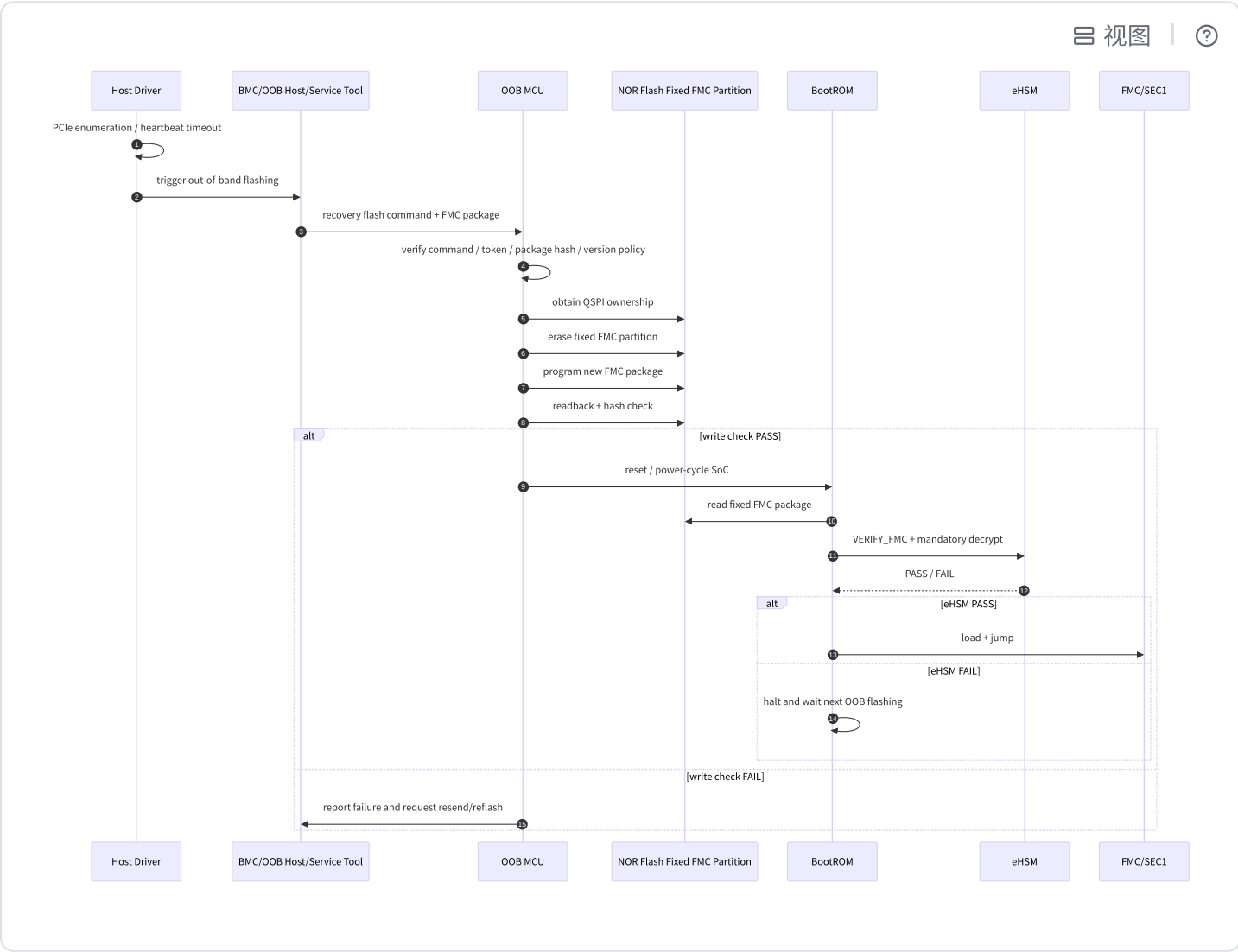
9.1 板级/OOB 安全架构

- 1. BMC/OOB Host 向 OOB MCU 发送管理、Debug、电源或刷机请求。
- 2. OOB MCU 在板级安全域内执行独立 secure boot 和授权检查。
- 3. 刷机时，OOB MCU 在可信环境内直接写 NOR Flash 固定 FMC 分区。
- 4. SoC 正常运行时，OOB MCU 可以把受控管理请求转发给 SEC/GSP。
- 5. OOB MCU 不访问 eHSM Root Key，不替代 eHSM 启动裁决。

9.2 OOB MCU 通过 QSPI 刷写 FMC

OOB MCU 刷机流程是独立于 SoC 正常运行态更新的带外流程。触发条件是 Host/BMC 发现卡无法连接、PCIe 枚举失败、心跳超时或运维/RMA/制造授权刷机。刷机过程不经过 SoC；SoC 只在下一次启动时验证刷入的 FMC。

图 OOB MCU QSPI 刷写与 SoC 启动验证关系



图下步骤说明：

1. Host 检测 PCIe 枚举失败或心跳超时。
2. Host/BMC/OOB Host 向 OOB MCU 下发 recovery flash command 和 FMC package。
3. OOB MCU 校验命令、token、包 hash 和版本策略。
4. OOB MCU 通过 QSPI 擦除、写入并读回校验固定 FMC 分区。
5. 复位 SoC 后，BootROM/eHSM 对新 FMC 执行 verify + mandatory decrypt，决定是否跳转。